CP372 Assignment 1 - Protocol Documentation

1. System Overview

This project is a text-based TCP client-server application written in Python. The system allows authenticated users to connect to a server, send messages, view available files, upload files, and download files using a custom application-layer protocol. The server handles one client at a time and stores uploaded files for future access.

Main Features

- User authentication
- Messaging between client and server
- File uploads
- File downloads
- File listing
- Checksum verification using SHA-256
- Error handling for invalid commands and transfers

---

2. Protocol Specification

The application uses a simple text-based protocol over TCP.

Connection Process

1. Client connects to the server.
2. User must authenticate with the LOGIN command.
3. After successful login, other commands become available.
4. Client may disconnect using the QUIT command.

Response Types

Success

```
OK <message>
```

Examples:

```
OK welcome Alice
OK READY
OK bye
```

Error

```
ERR <message>
```

Examples:

```
ERR file not found
ERR unknown user
```

---

3. Command Descriptions

LOGIN

Authenticates a user.

```
LOGIN Alice
```

The username must exist in the user list.

MSG

Sends a message to the server.

```
MSG Hello Server
```

The server acknowledges receipt.

LIST

Displays all files currently stored on the server.

```
LIST
```

UPLOAD

Uploads a file to the server.

```
UPLOAD hello.txt
```

The server receives and stores the file.

DOWNLOAD

Downloads a file from the server.

```
DOWNLOAD hello.txt
```

The downloaded file is saved locally.

QUIT

Closes the connection gracefully.

```
QUIT
```

HELP

Displays available commands.

```
HELP
```

RAW

Sends a raw protocol line for testing.

```
RAW <line>
```

---

4. Message Structure

All protocol messages are text-based and follow a command format.

General Structure

```
COMMAND [arguments]
```

Examples:

```
LOGIN Alice
MSG Hello Server
LIST
UPLOAD hello.txt
DOWNLOAD hello.txt
QUIT
```

Upload Sequence

```
Client: UPLOAD hello.txt
Server: OK READY
Client: <file data>
Server: OK <size> <checksum>
```

Download Sequence

```
Client: DOWNLOAD hello.txt
Server: OK <size> <checksum>
Server: <file data>
```

---

5. Design Decisions

TCP Communication

TCP was chosen because it provides reliable data transfer and ensures file contents arrive in the correct order.

Text-Based Protocol

The protocol uses plain-text commands because they are easy to implement, debug, and understand.

Username Authentication

Only users listed in the user database may log in, preventing unauthorized access.

SHA-256 Verification

Every file transfer includes checksum verification to ensure files are transferred without corruption.

Single-Client Server

The server handles one client at a time, simplifying protocol implementation and testing.

---

6. File Transfer Workflow

Upload Workflow

1. User issues the UPLOAD command.
2. Server responds with:

```
OK READY
```

1. Client sends the file data.
2. Server stores the file.
3. Server calculates a SHA-256 checksum.
4. Server sends confirmation and checksum information.

Download Workflow

1. User issues the DOWNLOAD command.
2. Server checks whether the file exists.
3. Server sends file information and checksum.
4. Server transfers the file data.
5. Client saves the file locally.
6. Client verifies the checksum.

Error Handling

The system handles:

- Unknown commands
- Login failures
- Missing files

- Unsafe filenames
- Invalid messages
- Mid-transfer disconnects

without crashing the client or server.

Group #6: Mike Bangar, Joshua Goudy, Anson Huang, Ali Lopez, Mahi Srivastav